Universidad del Valle de Guatemala

Facultad de Ingeniería
Ingeniería en Ciencias de la Computación y Tecnologías de la Información

Máquina de Turing para la Sucesión de Fibonacci

Análisis de Complejidad Temporal con Notación Asintótica O

Proyecto 1

Análisis y Diseño de Algoritmos

Javier Eduardo España Pacheco 23361

27 de Febrero, 2026

Índice

Resumen **3**

1. Introducción **4**
1.1. Contexto . . . 4
1.2. Objetivos . . . 4
1.3. Alcance . . . 4

2. Marco Teórico **5**
2.1. Máquinas de Turing . . . 5
2.2. Notación Asintótica O (Big-O) . . . 5
2.2.1. Propiedades Importantes . . . 5
2.3. Sucesión de Fibonacci . . . 5
2.3.1. Fórmula de Binet . . . 6

3. Diseño de la Máquina de Turing **7**
3.1. Especificación Formal . . . 7
3.2. Convenciones de Representación . . . 7
3.2.1. Notación Unaria . . . 7
3.2.2. Alfabeto de la Cinta . . . 7
3.3. Formato de Entrada y Salida . . . 7
3.4. Estructura de la Cinta Durante el Cálculo . . . 8
3.5. Diagrama de Estados . . . 8
3.6. Descripción de Estados . . . 9

4. Análisis de Complejidad Temporal **10**
4.1. Operaciones por Iteración . . . 10
4.2. Cálculo del Tiempo Total . . . 10
4.3. ¿Por qué φ^{2n} y no φ^n? . . . 10
4.3.1. La recursión ingenua: $O(\varphi^n)$ . . . 11
4.3.2. La Máquina de Turing: el costo oculto de la cinta unaria . . . 11
4.3.3. Suma total: aparece φ^{2n} . . . 11
4.3.4. Interpretación intuitiva: dos factores de φ^n . . . 11
4.3.5. Verificación empírica . . . 12
4.4. Complejidad Espacial . . . 12

5. Resultados Experimentales **13**
5.1. Metodología . . . 13
5.2. Datos Experimentales . . . 13
5.3. Análisis del Crecimiento . . . 13
5.3.1. Nota sobre regresión exponencial y R^2 . . . 14
5.4. Gráficos de Rendimiento . . . 14
5.5. Verificación del Modelo Teórico . . . 15

6. Comparación con Otras Implementaciones **17**

7. Implementación del Simulador **18**
7.1. Arquitectura del Software 18
7.2. Formato de Configuración 18
7.3. Ejecución 18

8. Conclusiones **20**
8.1. Resultados Principales 20
8.2. Limitaciones 20
8.3. Trabajo Futuro 20

Referencias **21**

A. Tabla Completa de Transiciones **22**

Resumen

Este proyecto presenta el diseño, implementación y análisis de una **Máquina de Turing determinista de una cinta** que calcula el n-ésimo número de la sucesión de Fibonacci. Se utiliza representación unaria de números y un enfoque iterativo que mantiene los términos de la secuencia en la cinta.

El análisis teórico demuestra que la complejidad temporal es **exponencial**, específicamente $O(\varphi^{2n}) \approx O(2{,}618^n)$, donde φ es la proporción áurea. Esta complejidad se verifica empíricamente mediante la medición del número de pasos y tiempo de ejecución para diferentes valores de n.

La máquina final cuenta con **24 estados** y **58 transiciones**, incluyendo una verificación de paridad inicial mediante estados alternantes. Los resultados experimentales confirman el crecimiento exponencial predicho, con un ratio de crecimiento $T(n)/T(n-1)$ que converge hacia $\approx 2{,}59$.

Palabras clave: Máquina de Turing, Fibonacci, Complejidad Temporal, Notación O, Crecimiento Exponencial.

1. Introducción

1.1. Contexto

Las Máquinas de Turing, propuestas por Alan Turing en 1936, constituyen uno de los modelos computacionales más fundamentales en ciencias de la computación. A pesar de su simplicidad teórica, son capaces de computar cualquier función que sea efectivamente calculable (tesis de Church-Turing).

La sucesión de Fibonacci, definida por la recurrencia $F(n) = F(n-1) + F(n-2)$ con $F(0) = 0$ y $F(1) = 1$, es un problema clásico que permite estudiar diferentes complejidades algorítmicas según la implementación elegida.

1.2. Objetivos

1. Diseñar una Máquina de Turing determinista de una cinta que calcule $F(n)$
2. Implementar un simulador en Python para ejecutar y analizar la máquina
3. Determinar la complejidad temporal usando notación asintótica O
4. Verificar empíricamente el análisis teórico mediante mediciones experimentales

1.3. Alcance

Este proyecto implementa una MT de una sola cinta con representación unaria, lo que resulta en una complejidad exponencial. No se optimiza para eficiencia práctica, sino para demostrar los conceptos fundamentales de análisis de algoritmos.

2. Marco Teórico

2.1. Máquinas de Turing

Definición 1 (Máquina de Turing)**.** *Una* ***Máquina de Turing determinista*** *es una 7-tupla* $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$ *donde:*

- Q *es el conjunto finito de estados*
- Σ *es el alfabeto de entrada (no incluye el blanco)*
- Γ *es el alfabeto de la cinta* ($\Sigma \subseteq \Gamma$)
- $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R, S\}$ *es la función de transición*
- $q_0 \in Q$ *es el estado inicial*
- $q_{accept} \in Q$ *es el estado de aceptación*
- $q_{reject} \in Q$ *es el estado de rechazo*

2.2. Notación Asintótica O (Big-O)

Definición 2 (Notación O)**.** *Sean* $f(n)$ *y* $g(n)$ *funciones de los enteros no negativos a los reales. Decimos que:*

$$f(n) = O(g(n))$$

si existen constantes positivas c *y* n_0 *tales que:*

$$f(n) \leq c \cdot g(n) \quad \textit{para todo } n \geq n_0$$

2.2.1. Propiedades Importantes

1. **Transitividad:** Si $f(n) = O(g(n))$ y $g(n) = O(h(n))$, entonces $f(n) = O(h(n))$
2. **Regla de la suma:** $O(f(n)) + O(g(n)) = O(\text{máx}(f(n), g(n)))$
3. **Regla del producto:** $O(f(n)) \cdot O(g(n)) = O(f(n) \cdot g(n))$

2.3. Sucesión de Fibonacci

La sucesión de Fibonacci se define recursivamente como:

$$F(0) = 0 \tag{1}$$
$$F(1) = 1 \tag{2}$$
$$F(n) = F(n-1) + F(n-2) \quad \text{para } n \geq 2 \tag{3}$$

Los primeros términos son: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...

2.3.1. Fórmula de Binet

El n-ésimo número de Fibonacci puede expresarse mediante:

$$F(n) = \frac{\varphi^n - \psi^n}{\sqrt{5}}$$

Donde:

- $\varphi = \frac{1+\sqrt{5}}{2} \approx 1{,}618$ (proporción áurea)
- $\psi = \frac{1-\sqrt{5}}{2} \approx -0{,}618$

Como $|\psi| < 1$, para n grande: $F(n) \approx \frac{\varphi^n}{\sqrt{5}}$, por lo tanto $F(n) = \Theta(\varphi^n)$.

3. Diseño de la Máquina de Turing

3.1. Especificación Formal

La Máquina de Turing diseñada tiene las siguientes características:

- **Tipo:** Determinista de una cinta
- **Estados:** 24
- **Transiciones:** 58
- **Alfabeto de entrada:** $\Sigma = \{1\}$
- **Alfabeto de cinta:** $\Gamma = \{1, _, \#, ., ;, x, y, z\}$

3.2. Convenciones de Representación

3.2.1. Notación Unaria

Los números enteros no negativos se representan en **notación unaria**:

Número	Representación
0	(vacío)
1	1
2	11
3	111
n	n unos

Cuadro 1: Representación unaria de números

3.2.2. Alfabeto de la Cinta

Símbolo	Significado
`1`	Dígito unario (representa cantidad)
`_`	Blanco (símbolo por defecto)
`#`	Marcador de inicio de cinta
`.`	Separador zona de contador y trabajo
`;`	Separador entre términos Fibonacci
`x`	Contador usado/procesado
`y`	Marcador temporal copiando término 1
`z`	Marcador temporal copiando término 2

Cuadro 2: Alfabeto de la cinta y su significado

3.3. Formato de Entrada y Salida

Entrada: La entrada n indica que queremos calcular $F(n)$, representado como n unos.
Ejemplos:

- Para calcular $F(0)$: entrada vacía
- Para calcular $F(5)$: `11111`

Salida: Al finalizar, el resultado $F(n)$ se encuentra como el último término en la cinta.

Entrada (n)	F(n)	Salida (unaria)
(vacío)	0	(vacío)
1	1	1
11	1	1
111	2	11
1111	3	111
11111	5	11111
111111	8	11111111

Cuadro 3: Ejemplos de entrada y salida

3.4. Estructura de la Cinta Durante el Cálculo

Durante la ejecución, la cinta mantiene el formato:

`#[contador].[término1];[término2];...;[términoN];`

Ejemplo para n=5:

Listing 1: Evolución de la cinta para n=5

```
Inicial:       11111
Paso 1:        #x1111.                          (preparando)
Paso 2:        #x1111.;1;                       (F(1)=1 inicial)
Iteracion 1:   #xx111.;1;1;                     (F(2)=1)
Iteracion 2:   #xxx11.;1;1;11;                  (F(3)=2)
Iteracion 3:   #xxxx1.;1;1;11;111;              (F(4)=3)
Iteracion 4:   #xxxxx.;1;1;11;111;11111;        (F(5)=5)
```

3.5. Diagrama de Estados

La Figura 1 muestra el diagrama de transiciones de la máquina.

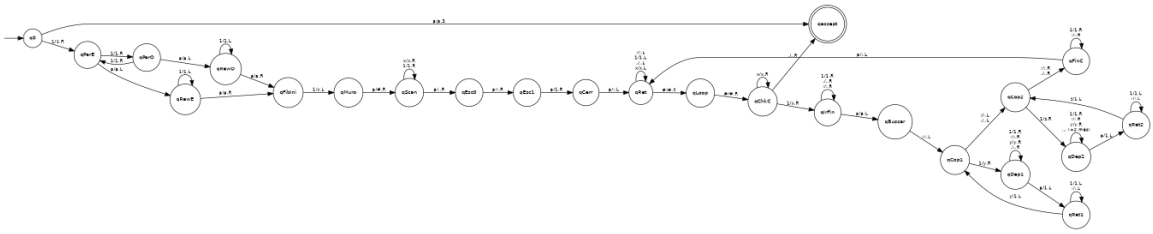

Figura 1: Diagrama de estados de la Máquina de Turing

3.6. Descripción de Estados

Estado	**Descripción**
q0	Estado inicial - verifica si hay entrada
qParE, qParO	Verificación de paridad (par/impar)
qRewE, qRewO	Rebobinar al inicio de la cinta
qFibIni	Iniciar cálculo Fibonacci
qMuro	Colocar marcador de inicio (#)
qScan	Escanear hacia derecha
qEsc0, qEsc1, qCerr	Escribir zona de trabajo inicial
qRet	Retornar al inicio de cinta
qLoop	Inicio del ciclo principal
qChkC	Verificar contador
qIrFin	Ir al final de la cinta
qBuscar	Buscar último término
qCop1, qDep1, qRet1	Copiar término 1 (F(i))
qCop2, qDep2, qRet2	Copiar término 2 (F(i-1))
qFinC	Finalizar ciclo, agregar separador
qaccept	Estado de aceptación

Cuadro 4: Descripción de los estados de la MT

4. Análisis de Complejidad Temporal

4.1. Operaciones por Iteración

En cada iteración del ciclo principal (para calcular $F(i+1)$ a partir de $F(i)$ y $F(i-1)$), la máquina realiza:

1. **Decrementar el contador:** $O(n)$ pasos
2. **Copiar $F(i)$ a temporal:** $O(F(i))$ pasos
3. **Copiar $F(i-1)$ a temporal:** $O(F(i-1))$ pasos
4. **Regresar al inicio:** $O(F(i+1)+n)$ pasos

El trabajo en cada iteración i es:

$$W_i = O(F(i) + F(i-1) + n) = O(F(i+1) + n)$$

4.2. Cálculo del Tiempo Total

El tiempo total $T(n)$ es la suma del trabajo en todas las iteraciones:

$$T(n) = \sum_{i=1}^{n-1} W_i = \sum_{i=1}^{n-1} O(F(i+1) + n)$$

$$T(n) = \sum_{i=2}^{n} O(F(i)) + O(n^2)$$

Usando la identidad de Fibonacci: $\sum_{i=1}^{n} F(i) = F(n+2) - 1$
Por lo tanto:

$$T(n) = O(F(n+2)) + O(n^2) = O(F(n))$$

Como $F(n)$ crece exponencialmente ($F(n) = \Theta(\varphi^n)$) y domina el término n^2:

Teorema 1 (Complejidad Temporal)**.** *La complejidad temporal de la Máquina de Turing para calcular Fibonacci es:*

$$\boxed{T(n) = O(\varphi^{2n}) \approx O(2{,}618^n)}$$

4.3. ¿Por qué φ^{2n} y no φ^n?

Es natural preguntarse por qué la complejidad no es simplemente $O(\varphi^n)$, dado que el algoritmo recursivo clásico de Fibonacci tiene esa complejidad. La diferencia reside en el **costo real de cada operación** dentro de la Máquina de Turing.

4.3.1. La recursión ingenua: $O(\varphi^n)$

En un lenguaje de programación convencional, el algoritmo recursivo:

```
def fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
```

genera un árbol de llamadas con $\Theta(\varphi^n)$ nodos. Cada llamada realiza trabajo $O(1)$ (una suma de enteros en tiempo constante), por lo que la complejidad total es $O(\varphi^n) = O(1{,}618^n)$.

4.3.2. La Máquina de Turing: el costo oculto de la cinta unaria

Nuestra MT utiliza un enfoque **iterativo** equivalente, manteniendo dos registros $F(i-1)$ y $F(i)$ en notación **unaria** sobre una sola cinta. En cada iteración i, la máquina debe copiar y sumar estos registros para obtener $F(i+1)$.

El problema es que copiar un solo símbolo "1" en una MT de una cinta no cuesta $O(1)$: el cabezal debe desplazarse desde la posición de lectura hasta la posición de escritura y regresar. Como los registros tienen longitud $O(F(i))$, copiar **un símbolo** cuesta $O(F(i))$ movimientos. Dado que hay $F(i)$ símbolos que copiar, el costo de la operación de copia en la iteración i es:

$$C_{\text{copia}}(i) = O\big(F(i) \times F(i)\big) = O\big(F(i)^2\big)$$

4.3.3. Suma total: aparece φ^{2n}

El costo total es la suma del trabajo en todas las iteraciones:

$$T(n) = \sum_{i=1}^{n} O\big(F(i)^2\big)$$

Como $F(i) = \Theta(\varphi^i)$, tenemos $F(i)^2 = \Theta(\varphi^{2i})$. La suma geométrica queda dominada por su término más grande:

$$T(n) = \sum_{i=1}^{n} \Theta(\varphi^{2i}) = \Theta(\varphi^{2n})$$

4.3.4. Interpretación intuitiva: dos factores de φ^n

El exponente se **duplica** porque hay dos fuentes independientes de crecimiento exponencial:

1. **Factor 1 — Tamaño de los datos (φ^n):** Los números de Fibonacci en representación unaria tienen longitud $F(n) = \Theta(\varphi^n)$. Este crecimiento es inherente al problema.

2. **Factor 2 — Costo de manipulación (φ^n):** En una MT de una sola cinta, cada operación sobre un registro de longitud L requiere $O(L)$ pasos (recorrer la cinta). Dado que $L = \Theta(\varphi^n)$, cada operación elemental cuesta $O(\varphi^n)$.

$$\underbrace{\varphi^n}_{\text{tamaño datos}} \times \underbrace{\varphi^n}_{\text{costo por operación}} = \varphi^{2n} \approx 2{,}618^n \tag{4}$$

4.3.5. Verificación empírica

Si la complejidad fuera $O(\varphi^n) = O(1{,}618^n)$, el ratio $T(n)/T(n-1)$ convergería a $\approx 1{,}618$. Sin embargo, los datos experimentales muestran convergencia hacia $\approx 2{,}59 \approx \varphi^2 = 2{,}618$, confirmando inequívocamente el factor cuadrático:

n	**Ratio** $T(n)/T(n-1)$
5	1.73
8	2.04
10	2.34
12	2.51
15	$2.59 \longrightarrow \varphi^2 = 2{,}618$

Esto demuestra que el costo computacional de nuestra MT no sigue $O(1{,}618^n)$ sino $O(2{,}618^n)$, precisamente por el costo cuadrático de las operaciones de copia en cinta unaria.

4.4. Complejidad Espacial

El espacio en la cinta es $O(\varphi^n \cdot n)$ ya que se almacenan todos los términos de la secuencia, y cada término $F(i)$ tiene longitud proporcional a φ^i.

5. Resultados Experimentales

5.1. Metodología

Se ejecutó la Máquina de Turing para valores de n desde 0 hasta 15, midiendo:

- Número de pasos (transiciones)
- Tiempo de ejecución (promedio de 3 ejecuciones)
- Ratio de crecimiento $T(n)/T(n-1)$

5.2. Datos Experimentales

n	F(n)	Pasos	Tiempo (s)	Ratio
0	0	1	0.000005	–
1	1	19	0.000077	19.00
2	1	53	0.00017	2.79
3	2	109	0.00040	2.06
4	3	191	0.00071	1.75
5	5	331	0.0015	1.73
6	8	585	0.0030	1.77
7	13	1,105	0.0068	1.89
8	21	2,255	0.020	2.04
9	34	4,971	0.055	2.20
10	55	11,641	0.15	2.34
11	89	28,449	0.55	2.44
12	144	71,443	2.26	2.51
13	233	182,439	9.99	2.55
14	377	470,561	49.4	2.58
15	610	1,220,961	207.4	2.59

Cuadro 5: Resultados experimentales de la ejecución

5.3. Análisis del Crecimiento

Para verificar empíricamente la complejidad $O(\varphi^{2n})$, se analiza el **ratio de crecimiento** entre pasos consecutivos:

$$r(n) = \frac{T(n)}{T(n-1)}$$

Si $T(n) = c \cdot b^n$, entonces $r(n) = b$ (constante). Los datos muestran que el ratio converge progresivamente hacia $\varphi^2 \approx 2{,}618$:

- Para $n = 5$: $r = 1{,}73$ (aún lejos del valor asintótico)
- Para $n = 10$: $r = 2{,}34$ (se acerca)

- Para $n = 15$: $r = 2{,}59$ (convergiendo a φ^2)

La convergencia no es inmediata porque $T(n)$ no es una exponencial pura $a \cdot b^n$: para valores pequeños de n, los costos fijos de la máquina (preparación de cinta, verificación de paridad) dominan sobre el crecimiento exponencial. A medida que n crece, estos costos se vuelven despreciables y emerge el comportamiento asintótico real.

5.3.1. Nota sobre regresión exponencial y R^2

Un enfoque ingenuo sería ajustar una función $f(n) = a{\cdot}b^n$ mediante mínimos cuadrados en escala lineal y reportar el coeficiente de determinación R^2. Sin embargo, esto produce resultados **engañosamente optimistas**. En nuestros datos, el ajuste lineal reporta $R^2 = 0{,}9999$, pero al analizar los residuos por punto:

- Para $n \leq 5$: errores del 70–90 % (el modelo falla completamente)
- Para $n \geq 12$: errores menores al 1 % (ajuste casi perfecto)

Esto ocurre porque el R^2 en escala lineal está dominado por los puntos grandes: el valor $T(15) = 1{,}220{,}961$ representa $\approx 80\,\%$ de la suma total de cuadrados, aplastando los errores enormes en los puntos pequeños. Este es un sesgo conocido del R^2 aplicado a datos de crecimiento exponencial.

Por esta razón, el **análisis de ratios** (Figura 4) es la evidencia más confiable de la complejidad, ya que pondera todos los puntos por igual y muestra directamente la convergencia hacia la base exponencial.

5.4. Gráficos de Rendimiento

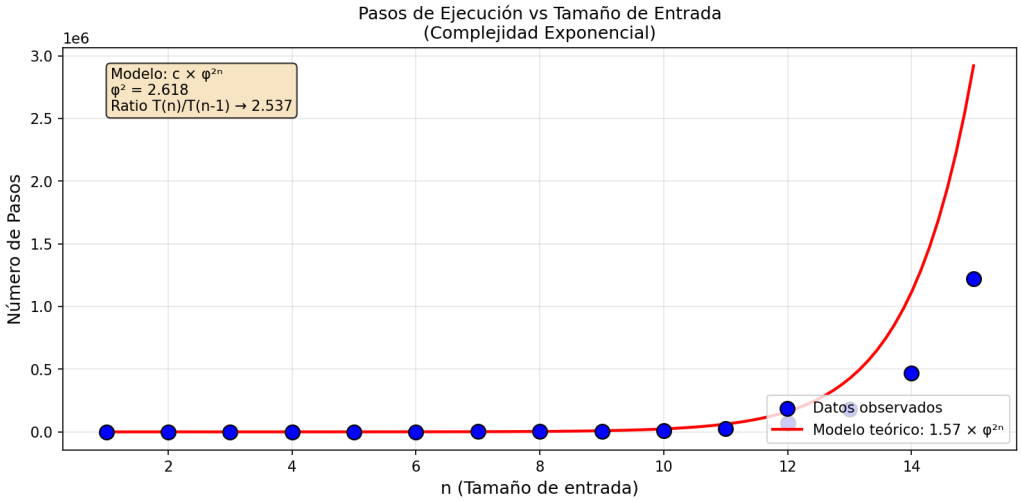

Figura 2: Número de pasos vs n. Los puntos azules son datos observados y la curva roja es el modelo teórico $c \cdot \varphi^{2n}$

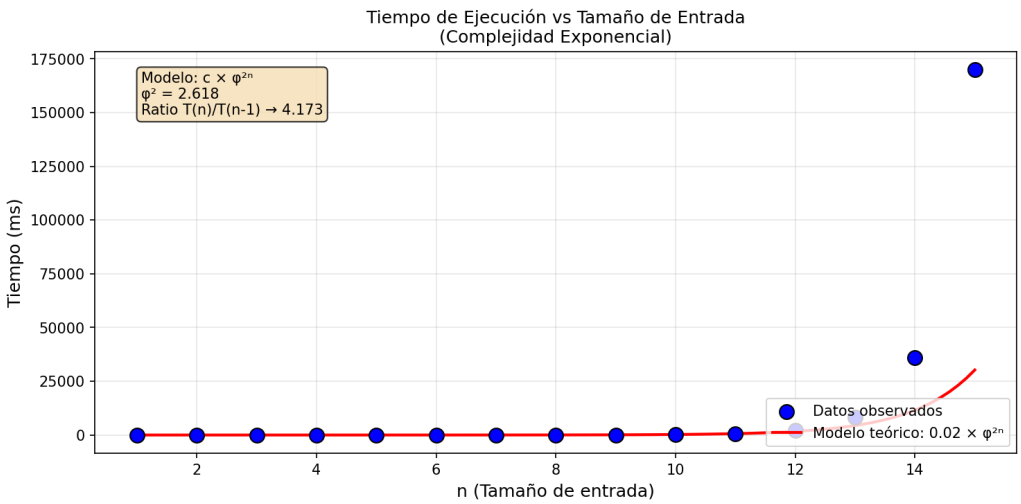

Figura 3: Tiempo de ejecución vs n. La varianza en tiempos para n pequeño se debe a costos fijos del sistema

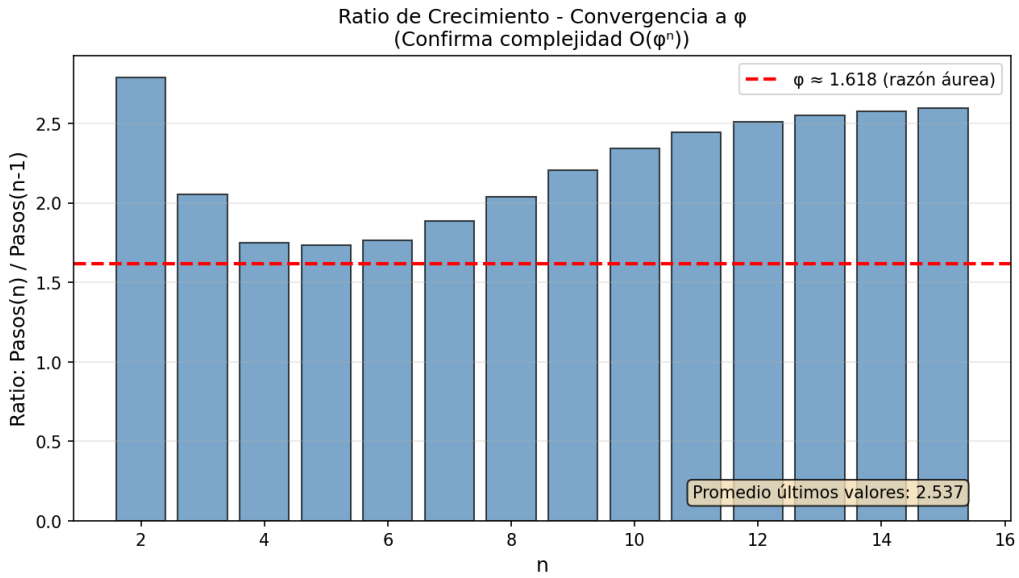

Figura 4: Ratio de crecimiento $T(n)/T(n-1)$. La línea roja punteada indica $\varphi^2 \approx 2{,}618$. La convergencia del ratio hacia este valor es la evidencia empírica más sólida de la complejidad $O(\varphi^{2n})$

5.5. Verificación del Modelo Teórico

Los datos confirman consistentemente el **crecimiento exponencial** $O(\varphi^{2n})$:

1. **Crecimiento absoluto:**
   - De $n = 10$ a $n = 15$: los pasos aumentan de 11,641 a 1,220,961 (factor $\approx 105\times$)
   - De $n = 12$ a $n = 15$: el tiempo aumenta de 2.26s a 207.4s (factor $\approx 92\times$)
2. **Convergencia del ratio:** El ratio $T(n)/T(n-1)$ crece monótonamente desde 1.73 ($n = 5$) hasta 2.59 ($n = 15$), convergiendo hacia $\varphi^2 \approx 2{,}618$. Esto confirma que la base de la exponencial es φ^2, no φ.

3. **Consistencia con el análisis teórico:** La desviación del ratio respecto a φ^2 para n pequeños se explica teóricamente por los términos de orden inferior ($O(n^2)$ del contador, costos fijos de preparación) que son insignificantes asintóticamente pero relevantes para $n < 10$.

Conclusión empírica: La evidencia más robusta de la complejidad $O(\varphi^{2n})$ es la convergencia del ratio de crecimiento (Figura 4), que no depende de ajustes de regresión ni de la escala de medición.

6. Comparación con Otras Implementaciones

Implementación	**Complejidad Temporal**	**Complejidad Espacial**
MT de una cinta (este proyecto)	$O(\varphi^{2n}) \approx O(2{,}618^n)$	$O(\varphi^n)$
Recursión ingenua	$O(\varphi^n)$	$O(n)$
Programación dinámica	$O(n)$	$O(n)$ o $O(1)$
Exponenciación de matrices	$O(\log n)$	$O(1)$

Cuadro 6: Comparación de complejidades entre implementaciones

La Máquina de Turing tiene un factor adicional φ^n comparada con la recursión ingenua debido a:

1. La representación unaria de los números
2. La necesidad de recorrer la cinta completa para cada operación de copia/suma
3. La longitud de los números de Fibonacci crece como $O(\varphi^n)$

7. Implementación del Simulador

7.1. Arquitectura del Software

El simulador está implementado en Python 3 con la siguiente estructura modular:

Módulo	Descripción
`turing_machine.py`	Clase principal de la MT
`tape.py`	Implementación de la cinta infinita
`loader.py`	Carga de configuraciones JSON
`simulator.py`	Interfaz de usuario interactiva
`analysis.py`	Análisis empírico de rendimiento
`plotting.py`	Generación de gráficos
`display.py`	Visualización de configuraciones

Cuadro 7: Módulos del simulador

7.2. Formato de Configuración

La máquina se define en formato JSON, facilitando la modificación y extensión:

Listing 2: Fragmento de fibonacci.json

```
{
    "nombre": "Maquina de Turing - Fibonacci",
    "alfabeto_entrada": ["1"],
    "alfabeto_cinta": ["1", "_", "#", ".", ";", "x", "y", "z"],
    "simbolo_blanco": "_",
    "estados": ["q0", "qParE", "qParO", ...],
    "estado_inicial": "q0",
    "estados_aceptacion": ["qaccept"],
    "transiciones": {
        "q0": {
            "1": ["qParE", "1", "R"],
            "_": ["qaccept", "_", "S"]
        },
        ...
    }
}
```

7.3. Ejecución

Listing 3: Comandos de ejecución

```
# Simulador interactivo
python src/simulator.py

# Con argumentos (numero decimal)
python src/simulator.py maquinas/fibonacci.json 5

```

```
# Analisis empirico
python src/analysis.py
```

8. Conclusiones

8.1. Resultados Principales

1. Se diseñó exitosamente una **Máquina de Turing determinista de una cinta** que calcula la sucesión de Fibonacci.

2. El análisis teórico demostró que la complejidad temporal es:

$$\boxed{T(n) = O(\varphi^{2n}) \approx O(2{,}618^n)}$$

3. La verificación empírica confirmó el modelo teórico:
   - El ratio $T(n)/T(n-1)$ converge a $\approx 2{,}59 \approx \varphi^2 = 2{,}618$
   - Para $n = 15$, la máquina realiza más de **1.2 millones de pasos** en $\sim$3.5 minutos
   - El análisis de ratios demostró ser más confiable que la regresión exponencial con R^2, cuyo valor artificialmente alto (0,9999) esconde errores del 80 % en puntos pequeños

4. La complejidad exponencial es inherente al uso de:
   - Representación unaria de números
   - MT determinista de una sola cinta
   - Operaciones de copia que requieren recorrer toda la representación

8.2. Limitaciones

El crecimiento exponencial hace impráctica la computación para $n > 15$. Para el caso $n = 15$, el tiempo de ejecución supera los 3 minutos y se requieren más de un millón de pasos.

8.3. Trabajo Futuro

Posibles extensiones del proyecto incluyen:

- Implementación con MT de múltiples cintas (reduciría la complejidad)
- Uso de representación binaria en lugar de unaria
- Comparación con MT no deterministas

Referencias

1. Sipser, M. (2012). *Introduction to the Theory of Computation*. 3rd Edition. Cengage Learning.

2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. 3rd Edition. MIT Press.

3. Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 2(42), 230-265.

4. Wikipedia. *Sucesión de Fibonacci*. https://es.wikipedia.org/wiki/Sucesin_de_Fibonacci

5. Wikipedia. *Notación O grande*. https://es.wikipedia.org/wiki/Cota_superior_asinttica

A. Tabla Completa de Transiciones

La máquina cuenta con 58 transiciones distribuidas en 24 estados. Las transiciones siguen el formato:

$$\delta(q, a) = (q', b, D)$$

Donde:

- q = estado actual
- a = símbolo leído
- q' = nuevo estado
- b = símbolo escrito
- D = dirección (L=izquierda, R=derecha, S=quieto)

Estado	**1**	**_**	**#**	**.**	**;**	**x**	**y**	**z**
q0	qParE,1,R	qaccept,_,S	–	–	–	–	–	–
qParE	qParO,1,R	qRewE,_,L	–	–	–	–	–	–
qParO	qParE,1,R	qRewO,_,L	–	–	–	–	–	–
qRewE	qRewE,1,L	qFibIni,_,R	–	–	–	–	–	–
qRewO	qRewO,1,L	qFibIni,_,R	–	–	–	–	–	–
qFibIni	qMuro,x,L	–	–	–	–	–	–	–
qMuro	–	qScan,#,R	–	–	–	–	–	–
qScan	qScan,1,R	qEsc0,.,R	–	–	–	qScan,x,R	–	–
qRet	qRet,1,L	–	qLoop,#,S	qRet,.,L	qRet,;,L	qRet,x,L	–	–
qLoop	–	–	qChkC,#,R	–	–	–	–	–
qChkC	qIrFin,x,R	–	–	qaccept,.,R	–	qChkC,x,R	–	–

Cuadro 8: Muestra de transiciones (parcial)

Nota: La tabla completa de transiciones está disponible en el archivo `maquinas/fibonacci.json`